

CEUB - Centro Universitário de Brasília



Disciplina: Projeto Integrador IV

Professora: Tiago Leite Pereira

Integrantes:

Fabricio Mendes - 22304559

Matheus Campos - 22301862

Luccas Vellasco - 22306479

Francisco Matheus - 22303388

GIP - Gestão Inteligente de Projetos

Equipe de Desenvolvimento

- **Fabricio:** Desenvolvimento e estruturação do **Frontend** (React.js, telas, navegação e integração de rotas).
- **Francisco:** Desenvolvimento e lógica do **Backend** (Node.js, Express, regras de negócio, autenticação e WebSockets).
- **Guilherme:** Modelagem do **Banco de Dados** e elaboração da **Documentação Técnica** (Prisma ORM, MER, validação de fluxo e relatórios).

1. Introdução

1.1 Apresentação do Projeto

O **GIP (Gestão Inteligente de Projetos)** é uma plataforma desenvolvida para facilitar a gestão de projetos, tarefas e equipes dentro de uma organização. O sistema utiliza quadros ágeis (Kanban), acompanhamento de desempenho e um sistema robusto de controle de acesso (RBAC).

1.2 Contexto do Problema

Organizações frequentemente enfrentam dificuldades em centralizar a comunicação, gerenciar permissões de forma granular e acompanhar o progresso das entregas. O uso de múltiplas ferramentas desconexas gera perda de informações, dificuldade em mensurar a performance da equipe e falta de visibilidade sobre o andamento dos projetos.

1.3 Justificativa

A criação desta plataforma justifica-se pela necessidade operacional de reduzir o tempo gasto no gerenciamento de times. Ao unificar o acompanhamento de tarefas, a comunicação em tempo real e as métricas de performance em um único ambiente seguro, a organização minimiza ruídos de comunicação, evita perda de prazos e otimiza a tomada de decisão gerencial embasada em dados precisos.

1.4 Público-Alvo

O sistema é voltado para corporações de médio e grande porte. Seus usuários variam desde a **alta gestão** (diretores e gestores de área), que necessitam de uma visão holística de desempenho (Dashboards), até a **camada operacional** (gerentes de projeto e funcionários), que utilizam o sistema no dia a dia para a execução das tarefas (Kanban).

1.5 Visão Geral da Solução Proposta

A plataforma é uma Single Page Application (SPA). O sistema possui controle de acesso com 5 níveis hierárquicos, quadros Kanban intuitivos (drag-and-drop), métricas de desempenho e comunicação integrada (Chat) funcionando em tempo real (via WebSockets) para manter todas as partes interessadas alinhadas.

2. Objetivos

2.1 Objetivo Geral

Fornecer um ambiente digital centralizado e seguro para o planejamento, execução e monitoramento de projetos e tarefas, otimizando a produtividade das equipes.

2.2 Objetivos Específicos

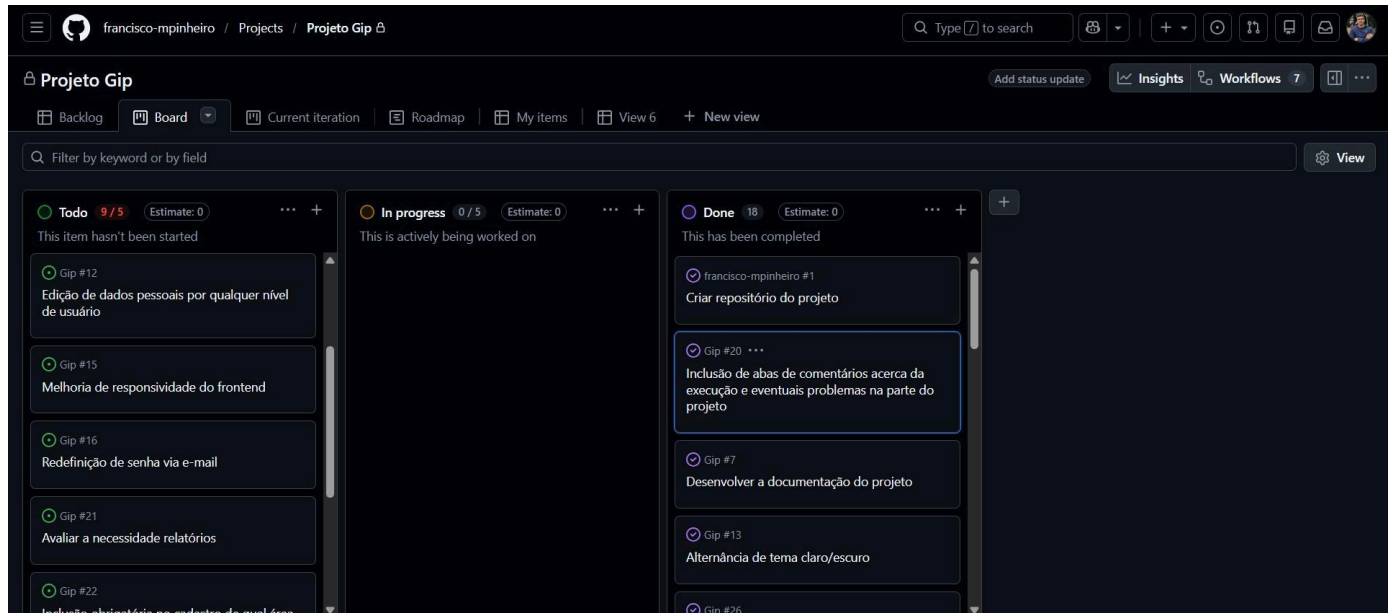
- Centralizar o cadastro e a gestão de projetos e tarefas (CRUD).
- Controlar acessos e permissões granulares por meio de RBAC (Role-Based Access Control).
- Oferecer visibilidade do progresso do trabalho via Kanban interativo.
- Monitorar a performance e atividades das equipes.
- Facilitar a comunicação através de um módulo de chat interno em tempo real.

3. Metodologia

O ciclo de desenvolvimento do software foi pautado por metodologias ágeis, focando em entregas iterativas e incrementais. Todo o planejamento, organização de épicos (Epics) e distribuição de tarefas (Issues) foi gerenciado através de um quadro de **Backlog no GitHub Projects**.

O repositório do GitHub também serviu como ambiente centralizador do versionamento de código, garantindo rastreabilidade das alterações e facilitando revisões.

Quadro de Backlog



4. Tecnologias Utilizadas

A escolha das tecnologias (Stack) visou garantir escalabilidade, segurança e uma interface moderna e responsiva.

- **Frontend (React.js):** Escolhido por permitir a criação de uma SPA (Single Page Application) dinâmica e reativa, onde componentes como o Kanban podem ter seus estados atualizados localmente antes mesmo da resposta do servidor, garantindo uma usabilidade fluida (sem *reloads* de página).
- **Backend (Node.js + Express):** Adotado pela alta capacidade de lidar com múltiplas conexões assíncronas (I/O não-bloqueante) e por compartilhar a mesma linguagem (JavaScript) do frontend.
- **Banco de Dados (PostgreSQL):** Selecionado por ser um banco relacional robusto, ideal para estruturar hierarquias complexas (usuários, projetos, roles) e manter integridade de chaves estrangeiras.
- **ORM (Prisma):** Utilizado para intermediar a comunicação com o banco de dados. O Prisma oferece tipagem forte, autocompletes de queries e mitiga drasticamente riscos de *SQL Injection*.
- **Comunicação em Tempo Real (Socket.io):** Integrado ao Node.js para permitir que o módulo de Chat dispare mensagens ativamente para os clientes conectados, eliminando a necessidade de *long-polling*.

5. Regras de Negócio

RN01 - Níveis de Acesso Hierárquicos (RBAC)

O sistema opera com 5 papéis (Roles) bem definidos:

- **Admin Plataforma (`admin_platform`)**: Acesso irrestrito (gerencia usuários, projetos de todas as empresas e exclui entidades globalmente).
- **Admin Empresa (`admin_company`)**: Gerencia todos os recursos da sua empresa. Não pode excluir outros usuários do sistema de forma permanente, apenas inativá-los.
- **Gestor de Área (`manager_area`)**: Cria projetos e tarefas, vê todos os projetos de sua área, mas não gerencia usuários do sistema nem exclui projetos.
- **Gerente de Projeto (`project_manager`)**: Vê apenas projetos dos quais participa ou é o gerente criador. Pode atribuir tarefas aos membros e visualizar o desempenho do projeto.
- **Funcionário (`employee`)**: Vê apenas projetos dos quais é membro. Apenas interage com as tarefas que lhe foram atribuídas (podendo movê-las no Kanban).

RN02 - Visibilidade e Acesso a Projetos

Um usuário do nível operacional só pode visualizar, buscar e acessar projetos em que está designado como membro ou gestor, mantendo o isolamento da informação.

RN03 - Movimentação de Tarefas

Apenas o usuário atribuído (`assignee`), o gerente do projeto, gestores superiores ou admins podem alterar o status de uma tarefa.

RN04 - Comunicação em Tempo Real

Os usuários podem interagir via chat. A API do chat utiliza WebSockets (`socket.io`) para entrega instantânea das mensagens.

RN05 - Registro de Auditoria (Logs de Atividade)

Toda mudança de estado em Projetos e Tarefas deve registrar automaticamente uma "Activity" vinculada ao usuário responsável pela ação.

6. Requisitos

6.1 Requisitos Funcionais

6.2

ID	Descrição	Nível Mínimo
RF01	Permitir o cadastro, login e autenticação de usuários via JWT.	Todos
RF02	Permitir o CRUD de usuários (Cadastro, Edição, Inativação).	Admin
RF03	Permitir a criação e edição de Projetos.	Gestor
RF04	Permitir a alocação de usuários aos Projetos.	Gestor / Gerente
RF05	Permitir a criação, edição e atribuição de Tarefas num Projeto.	Gestor / Gerente
RF06	Fornecer quadro Kanban para gestão das tarefas.	Autenticado
RF07	Exibir Dashboard consolidado.	Autenticado
RF08	Prover painel de Performance das equipes.	Gerente+
RF09	Permitir o gerenciamento do Perfil do usuário.	Autenticado
RF10	Prover Chat integrado para mensagens entre equipe (via WebSocket).	Autenticado
RF11	Prover central de notificações para alertas sobre tarefas e projetos.	Autenticado
RF12	Permitir a redefinição de senha através do envio de e-mail de recuperação.	Todos
RF13	Permitir o upload de certificados e a gestão de formação acadêmica no perfil.	Autenticado

6.3 Requisitos Não Funcionais

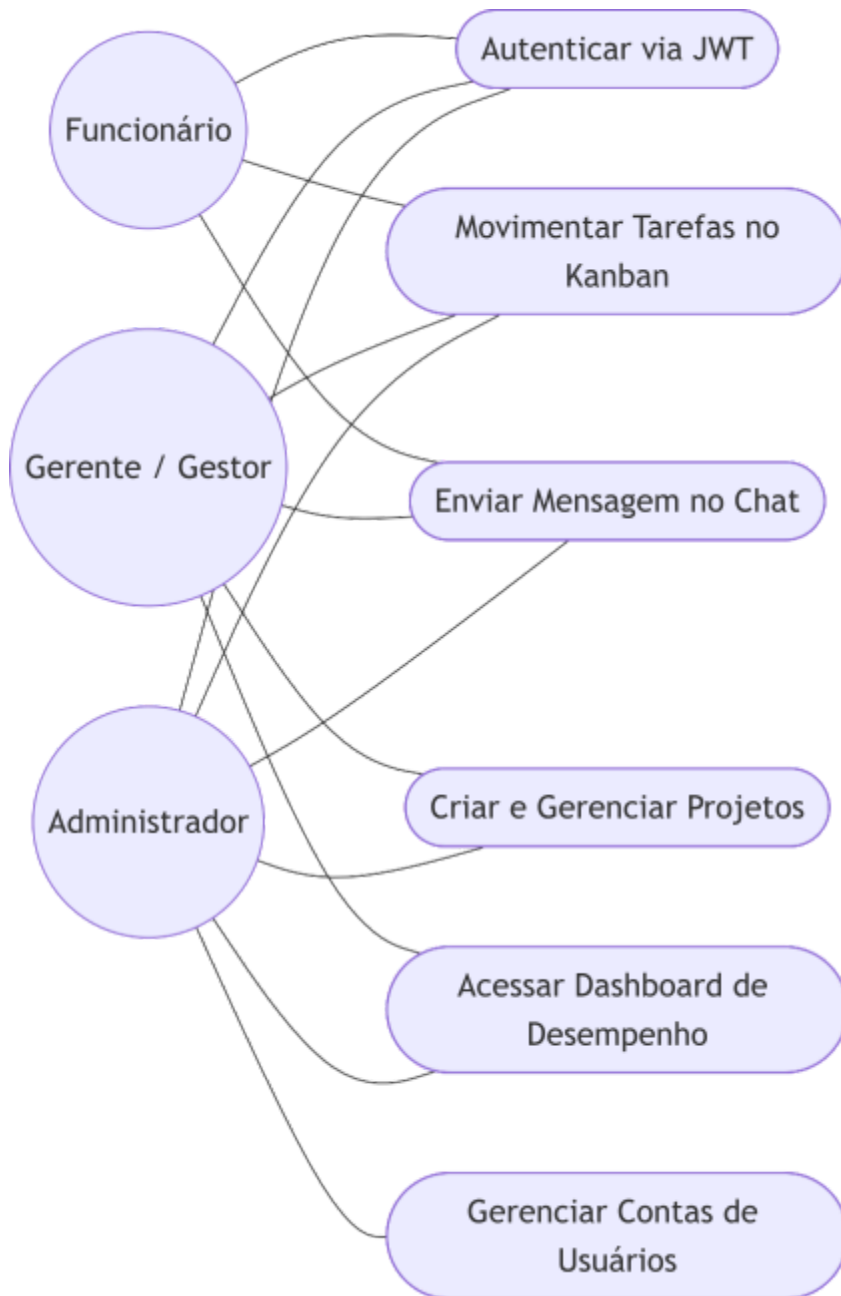
ID	Descrição	Categoria
RNF01	A interface é uma SPA construída em React.js.	Arquitetura
RNF02	O backend é uma API RESTful em Node.js com Express.	Arquitetura
RNF03	O banco de dados é PostgreSQL gerenciado pelo Prisma ORM.	Banco de Dados

RNF04	A autenticação utiliza tokens JWT nos cabeçalhos (Bearer).	Segurança
RNF05	As senhas são protegidas por hash no armazenamento.	Segurança
RNF06	A comunicação em tempo real é viabilizada por Socket.io .	Arquitetura

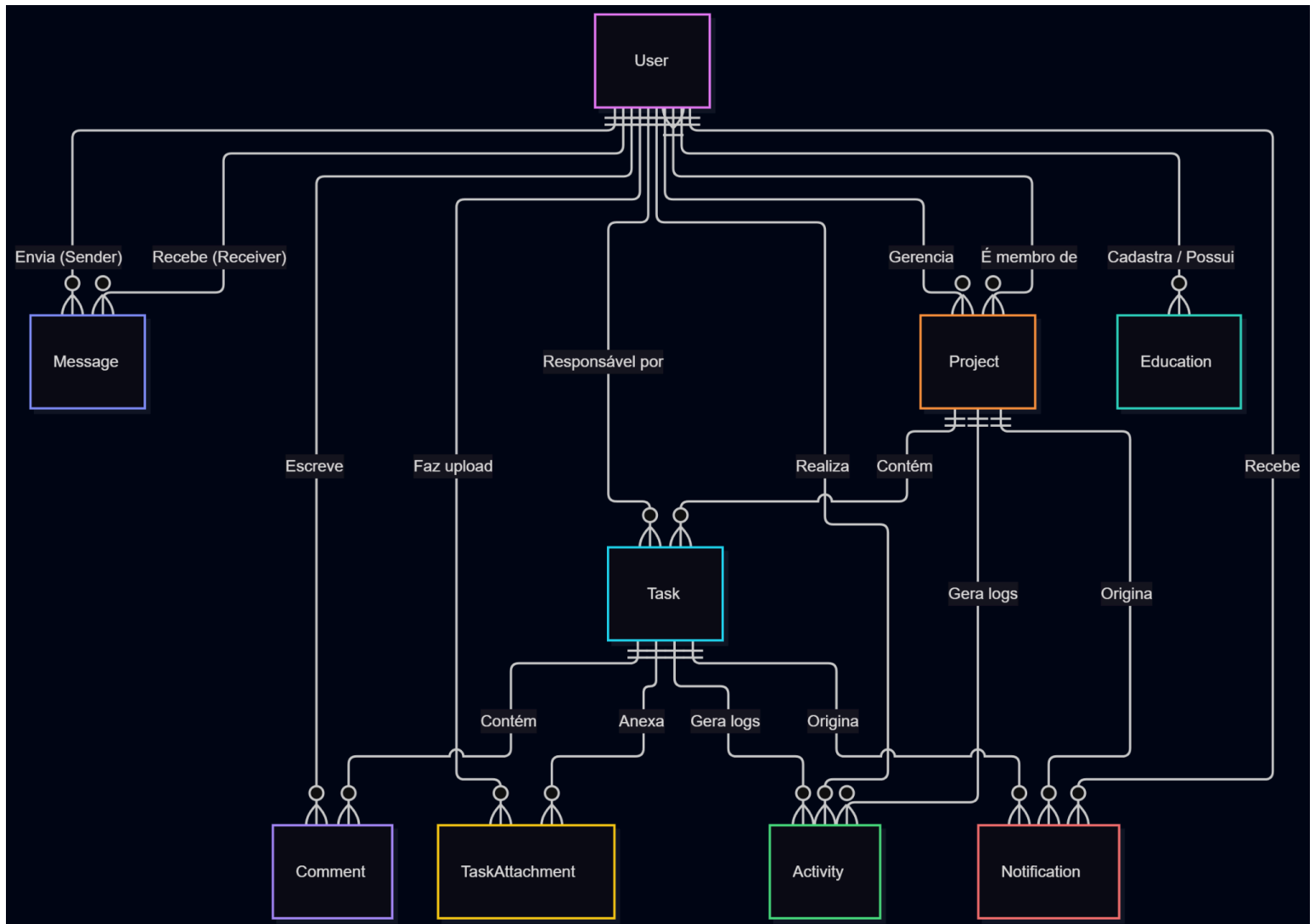
7. Diagramas e Modelos

7.1 Diagrama de Casos de Uso (UML)

O diagrama abaixo mapeia as interações já existentes dos diferentes perfis de usuários com o sistema:



7.2 Diagrama MER (Mermaid)



7.3 Entidades e Valores Padrões (Prisma)

Abaixo está o reflexo **exato** do que existe no `schema.prisma` atualmente implementado:

- **Role (Enum explícito no banco de dados):**
`admin_platform` , `admin_company` , `manager_area` , `project_manager` , `employee` .
- **Status do Projeto (`Project.status`):** Campo do tipo `String` (padrão: `"planejamento"`).
- **Status da Tarefa / Kanban (`Task.status`):** Campo do tipo `String` (padrão: `"a_fazer"`).

8. Arquitetura do Sistema

O ecossistema baseia-se no modelo Client-Server:

1. **Frontend (Client):** Renderizado no navegador via **React.js**. O React gerencia rotas e

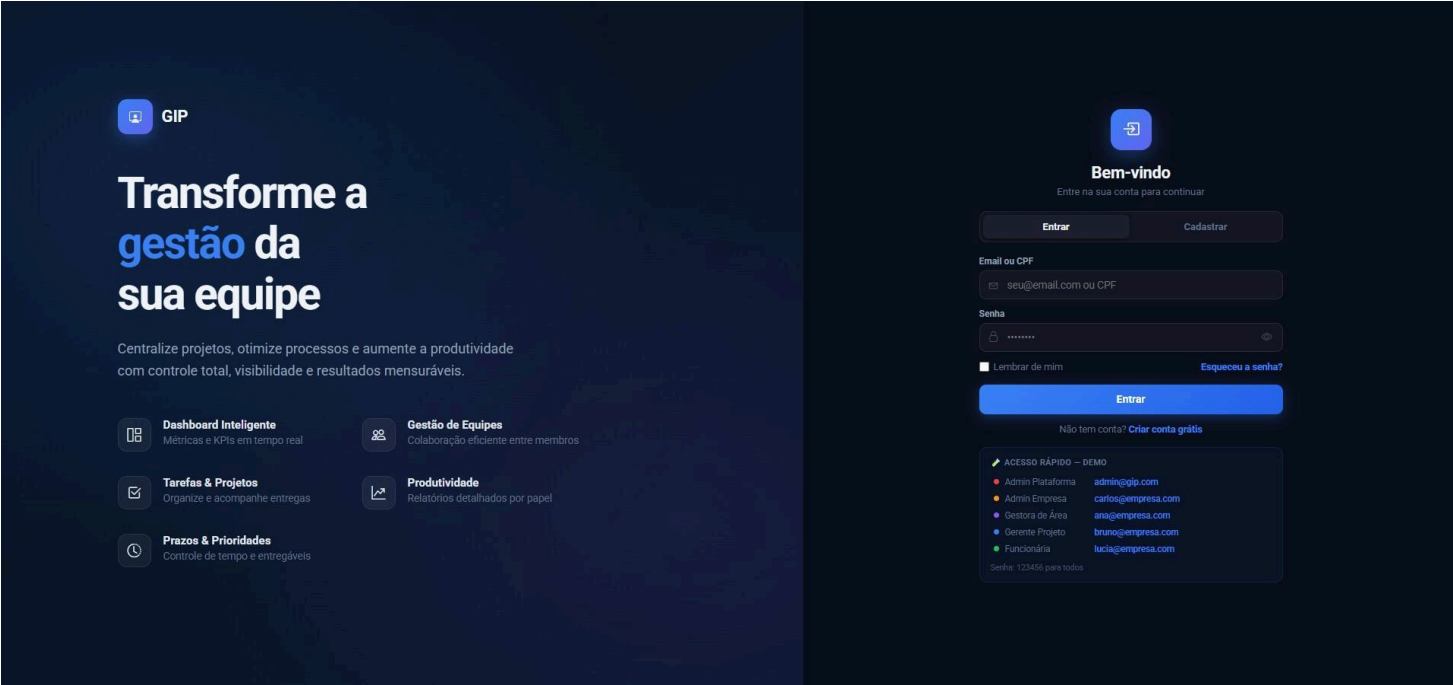
estados. Todo acesso a dados é repassado pela interface `api.js` (Axios) portando o token JWT.

2. **Backend (Server): Node.js** com **Express** atuando puramente como provedor de API (JSON), e **Socket.io** para comunicação síncrona.
3. **Database (Data Layer): PostgreSQL** mediado pelo **Prisma ORM**.

9. Especificação de Telas Implementadas

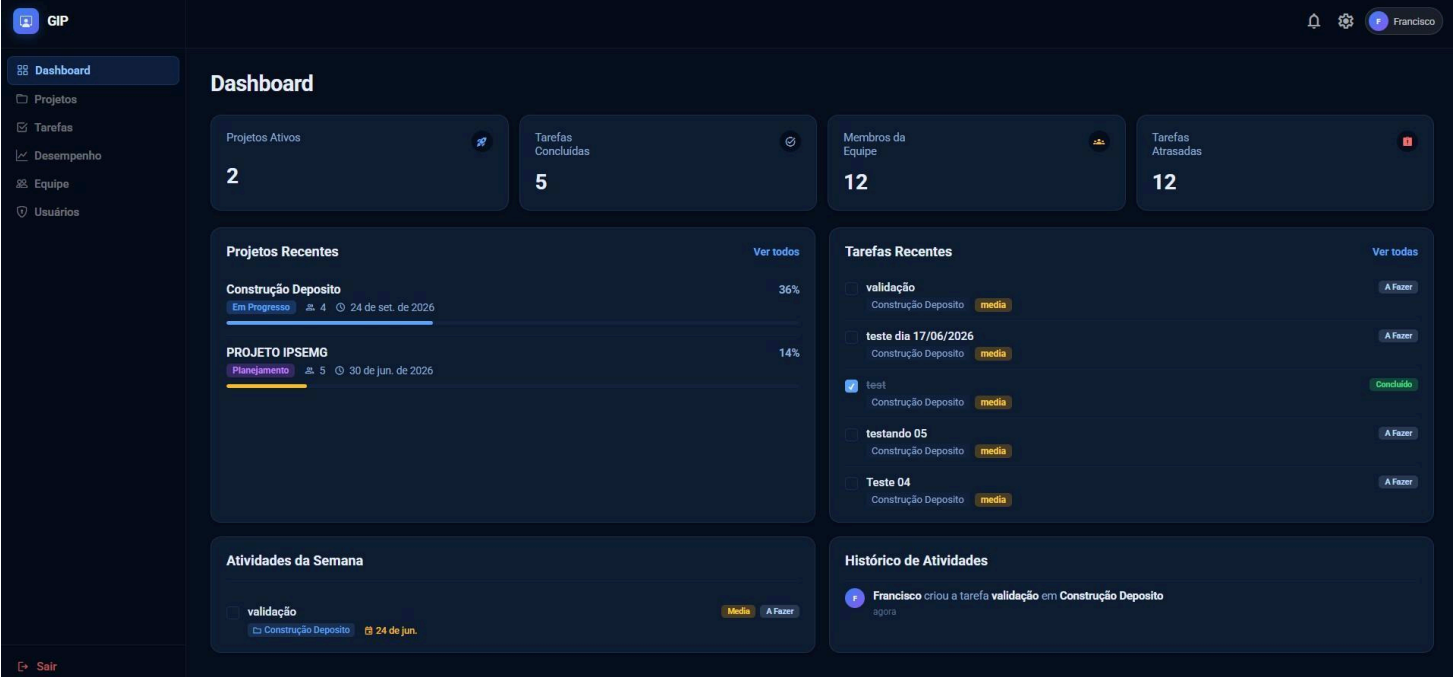
9.1 Tela de Login (/login)

Ponto de entrada do sistema onde os usuários se autenticam.



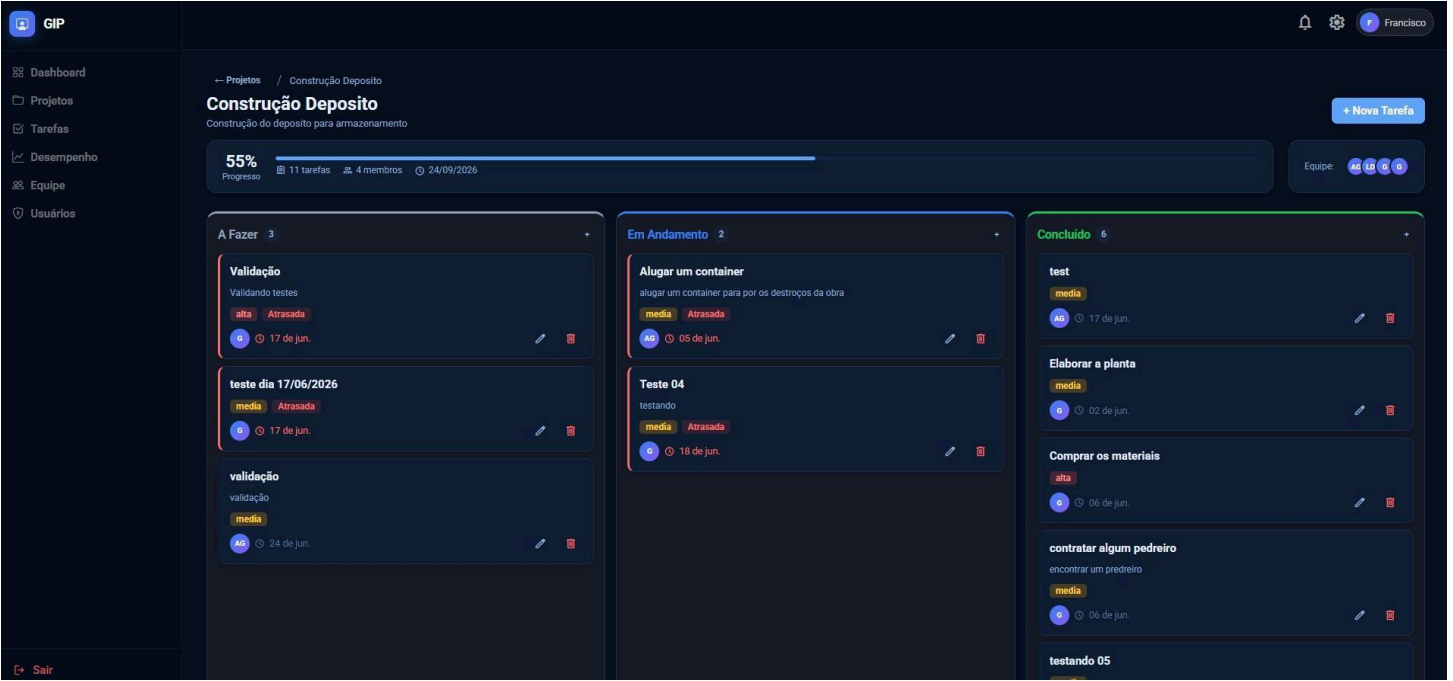
9.2 Dashboard Page (/dashboard)

Estatísticas rápidas, gráfico de entregas, feed de atividades recentes.



9.3 Kanban / Project Detail (/projects/:id)

Colunas verticais para os status da tarefa. Arrastar e soltar cartões.

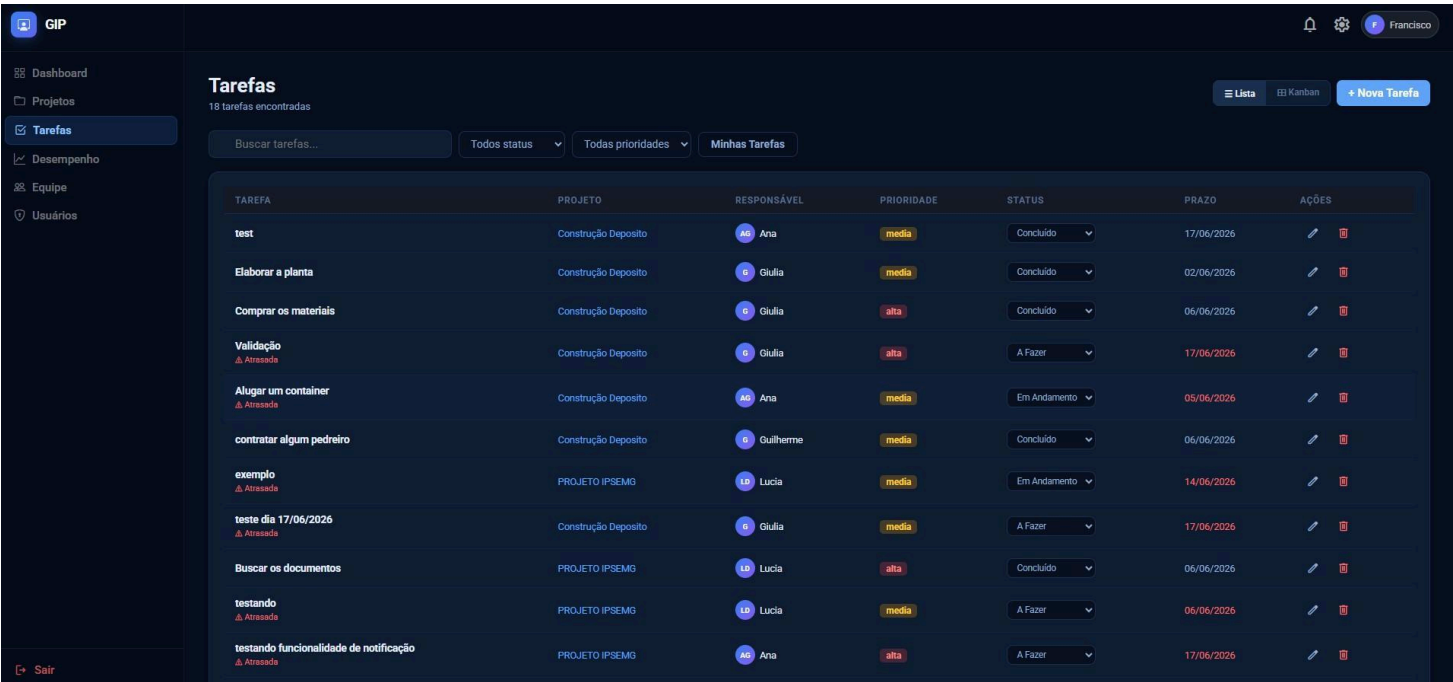


9.4 Gestão de Formação Acadêmica (/profile/academic)

Funcionalidade que permite ao usuário realizar o upload de certificados e gerenciar suas formações acadêmicas diretamente no seu perfil.

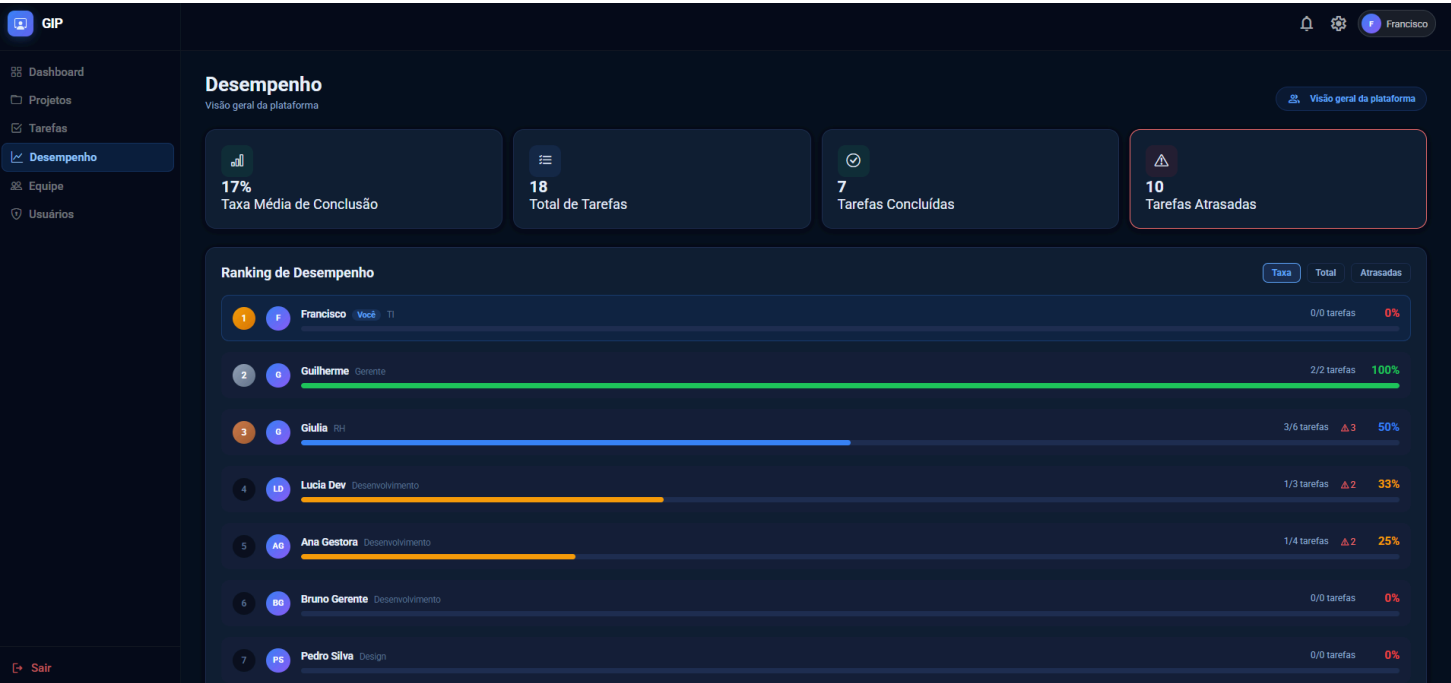
9.5 Tela de Tarefas

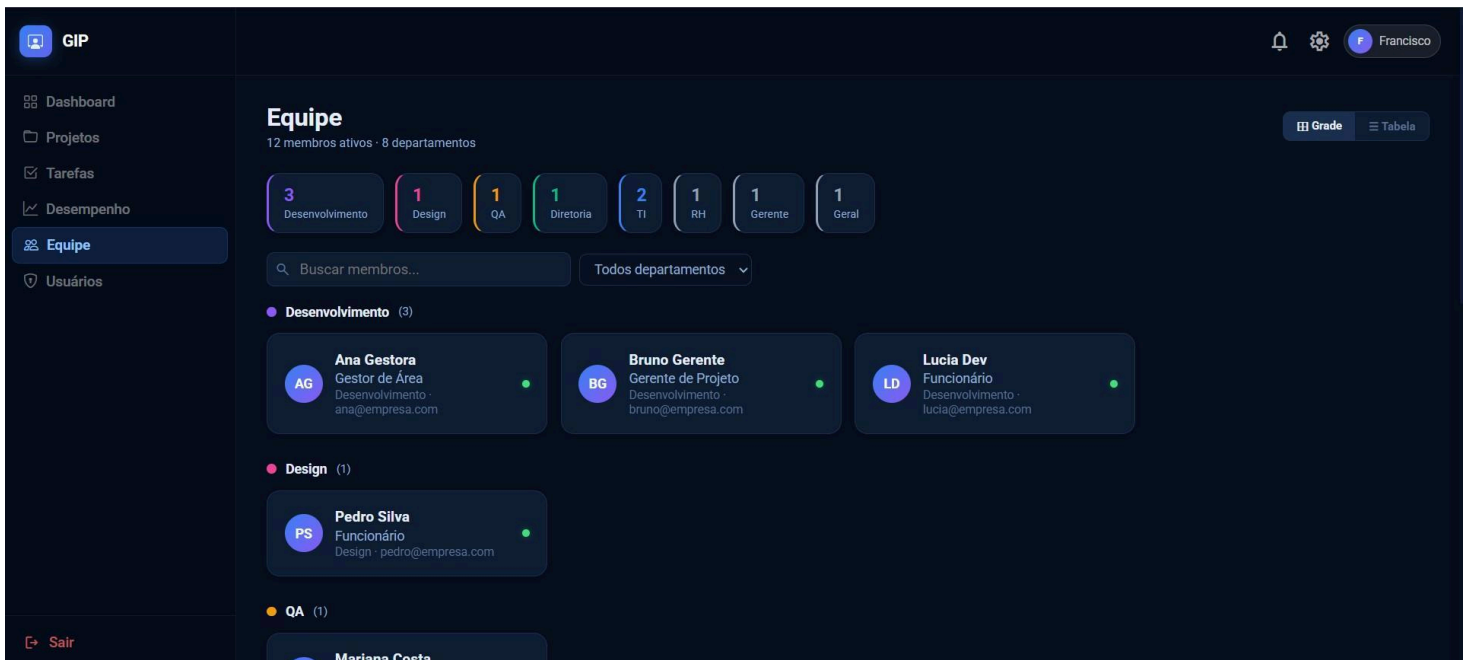
Visualização e gestão das tarefas do sistema.



9.6 Performance Page (/performance)

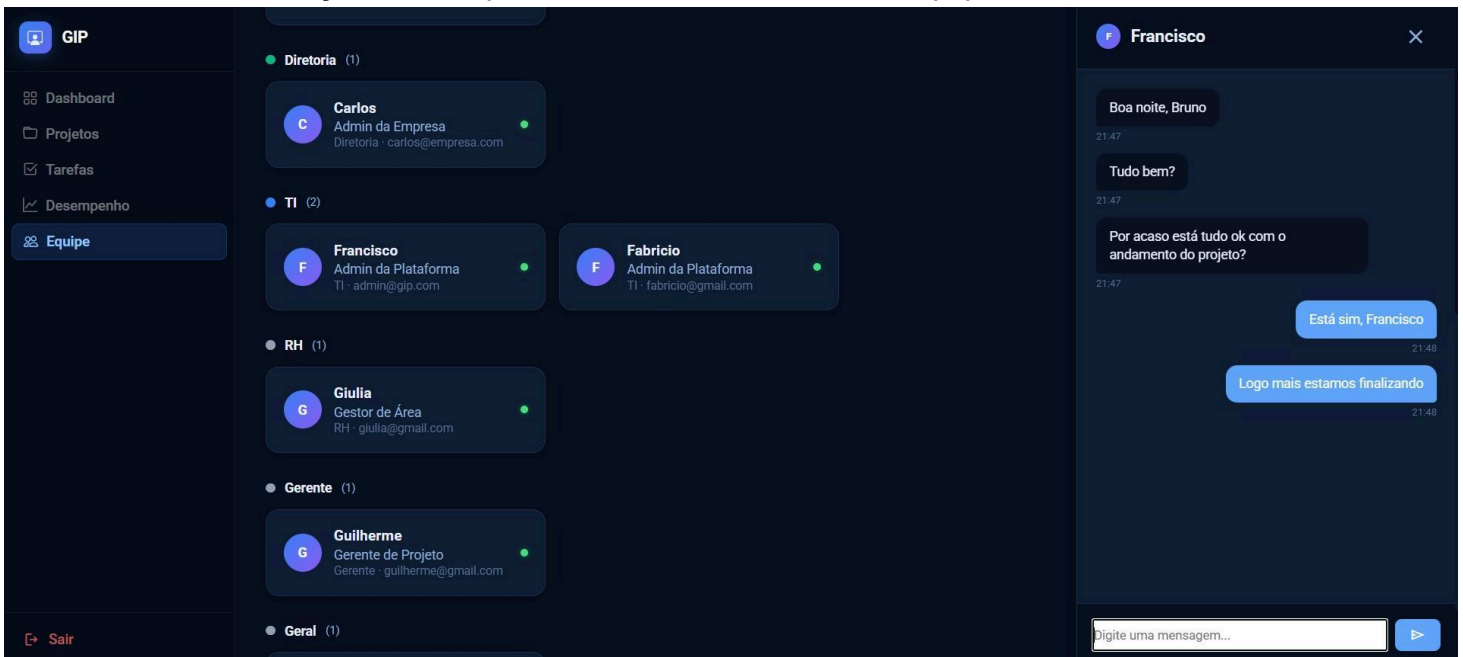
Ranking de tarefas finalizadas. Visualizável por gestores, gerentes e admins.





9.9 Chat da Equipe

Interface de comunicação em tempo real entre os membros da equipe.



10. Documentação da API

10.1 Padrão de Comunicação e Tratamento de Erros

A API utiliza o formato JSON. Todas as rotas (exceto de registro e login) exigem cabeçalho:

Authorization: Bearer <token_jwt> . Quando ocorre uma exceção, o *Global Error Handler* do servidor

responde no formato:

```
{
  "message": "Erro interno do servidor",
  "error": "Detalhe da mensagem de erro",
  "stack": "Stack trace (em modo dev)"
}
```

10.2 Principais Rotas

- **Auth:** POST /api/auth/login , POST /api/auth/register , GET /api/auth/me .
- **Password Reset:** POST /api/auth/forgot-password , POST /api/auth/reset-password .
- **Users:** GET /api/users , POST /api/users , PUT /api/users/:id .
- **Projects:** GET /api/projects , POST /api/projects , PUT /api/projects/:id .
- **Tasks:** GET /api/tasks , POST /api/tasks , PATCH /api/tasks/:id/status .
- **Chats:** GET /api/chats/:chatId/messages , POST /api/chats/:chatId/messages .
- **Dashboard & Performance:** GET /api/dashboard, GET /api/performance.
- **Notificações:** GET /api/notifications, PATCH /api/notifications/:id/read.
- **Educação / Formação Acadêmica:** GET /api/education, POST /api/education.

11. Autenticação e Segurança

- **Autenticação baseada em JWT único:** Ao realizar o login, a API devolve um JWT. Este token deve ser incluído no cabeçalho Authorization: Bearer <token> nas requisições.
- **Armazenamento de Senhas:** As senhas dos usuários utilizam *hashing* no backend antes de serem salvas no PostgreSQL, garantindo que não sejam armazenadas em texto puro.

12. Conclusão e Trabalhos Futuros

A documentação atual representa estritamente os artefatos, fluxos, telas e arquitetura que já estão integrados e funcionais no repositório do GIP (Gestão Inteligente de Projetos). O MVP foi concluído com sucesso ao entregar o controle de acessos (RBAC), o gerenciamento através de Kanban e as interações dinâmicas alimentadas por WebSockets.

12.1 Trabalhos Futuros

Apesar do sistema cobrir as necessidades do MVP, como próximos passos evolutivos (*roadmap*), sugerem-se as seguintes implementações técnicas:

- **Integração de E-mails Transacionais:** Para envio de convites de cadastro e fluxos de recuperação de senha.
- **Armazenamento em Nuvem (Cloud Storage):** Refatorar o sistema atual de "Anexos" (que utiliza armazenamento local em disco na pasta `/uploads`) para provedores de nuvem profissionais, como AWS S3 ou Google Cloud Storage, visando escalabilidade.
- **Middlewares de Validação de DTOs:** Implementar bibliotecas como Zod ou Yup para validação estrita de requisições antes que atinjam os *Controllers*.